# OPALA: Optimized Pruning Adaptive Learning Approach for Federated Learning Scenarios

R. Veiga*, R. Morais*, R. Bustincio†, L. Bastos*, D. Rosário*, S. Sargento‡, and E. Cerqueira*
* Dept. of Computer Science - Federal University of Pará, Brazil
† State University of Campinas (UNICAMP), Brazil
‡ Instituto de Telecomunicações, University of Aveiro, Portugal
Emails: rafael.teixeira.silva@icen.ufpa.br, romulo.bustincio@ic.unicamp.br
{renan.morais,lucas.bastos}@itec.ufpa.br, susana@ua.pt, {denis,cerqueira}@ufpa.br.

***Abstract*—Federated learning (FL) enables decentralized model training, allowing data privacy by processing locally on client devices. However, challenges such as high communication costs, especially in heterogeneous environments, hinder the efficient scaling of FL systems. This paper introduces OPALA, an adaptive pruning optimal learning approach designed to improve personalized Federated Learning (pFL) efficiency and accuracy. OPALA combines dynamic pruning with adaptive aggregation, adjusting model complexity based on the client's computational capabilities and data characteristics. In this sense, OPALA significantly reduces the processing time of clients to train their models based on the computational capabilities of each client, providing higher system efficiency and responsiveness to real-time applications. The results show that OPALA significantly improves training time and data transmission by up to 53% without sacrificing the model accuracy, outperforming traditional methods. This approach offers a promising solution for efficient, scalable FL in environments with non-IID data and heterogeneous client resources.**

***Index Terms*—Federated Learning, Pruning, Adaptive, Personalized**

## I. Introduction

Rapid expansion of intelligent systems created a pressing need for Machine Learning (ML) models capable of adapting to diverse environments [1]. Meanwhile, researchers highly appreciate a distributed learning paradigm known as Federated Learning (FL), which is defined as an ML environment in which multiple devices collaborate to train a global model collaboratively without sharing their local data under the coordination of a central server [2]. In this way, FL reduces transmission overhead while providing private information by sharing model parameters instead of raw data [3].

In traditional FL environments, clients train and transmit complete model updates, which could have millions of parameters that lead to a larger model size [4]. For example, ResNet could have more than 25 million parameters, leading to a considerable gradient and a large model size [5]. However, client devices might have heterogeneous computing capabilities, ranging from low-end smartphones to edge devices with dedicated GPUs, such as a Nvidia Jetson [6]. In addition, clients rely on different communication technologies (*e.g.*, 4G, 5G, Wi-Fi, etc.) to distribute the updated model with the aggregation server. In this heterogeneous scenario, the server must wait for slow learners (a.k.a. stragglers) in each communication round, where such clients delay the overall execution of FL [6]. In this way, it is not practical to train the complete ML model for all clients without dealing with computational constraints, and also, disseminating the complete model through the network could be prohibitive [7].

Exploring methods to reduce the model size to optimize network communication is crucial, as model training creates a bias due to its redundancy of data concerning many model parameters that are not crucial for the entire neural network [8]. This leads to unnecessary processing and communication costs [9]. In this context, personalized Federated Learning (pFL) tailors models to clients' specific needs to combat client heterogeneity and provides a client-specialized model [10]. After training, each client develops a model that captures the distinct characteristics of its data, adjusting the model according to the importance of the layer to better fit its unique characteristics [11]. The pFL approach allows stragglers to contribute to the model aggregation process without compromising the training. In the pFL context, the amount of relevant parameters in each layer indicates its relevance for the client model, which means that it measures both potential loss and gains in the model's use of parameters, indicating how different the model is from the trained result [12].

Pruning methods remove unnecessary weights from the model parameters to reduce the communication cost by transmitting fewer gradients to the central server [13]. Conversely, an unstructured prune removes individual weight elements, resulting in a sparse network with many zero values. However, maintaining sparse data structures requires extra storage for information like compressed sparse rows or binary masks. On the other hand, a structured prune removes entire filters rather than individual weights, maintaining the dense structure of the global model while ensuring compatibility with the simple computational capabilities of client devices. In this sense, it is important to use structural pruning based on the client's parameters and their layers to reduce training and transmission time, allowing to cut the model size and its weights to reduce both the training time and model transmission. However, existing methods do not provide adaptability to adjust pruning based on a device's processing power or data, enabling slower devices to contribute to the global model [14]. In addition, some existing methods often use a one-size-fits-all model

for every device, which does not work well for devices with different processing capabilities, making computation and communication less efficient. Hence, it is important to design an adaptive structural pruning approach to reduce the model size based on data and client context information to enable clients with less data or slower processing capabilities to contribute to the global model with faster convergence, which is still an open issue.

In this paper, we introduce an adaptive structural pruning approach to enhance the generalization capabilities of the pFL model, called OPALA. In its operation, OPALA combines dynamic pruning with adaptive aggregation, adjusting the complexity of the model based on the processing time, training time, and characteristics of the data size. Specifically, OPALA improves the model performance by enabling clients who cannot send their parameters during training processing or do not have enough power to train. In this sense, OPALA reduces the processing time to train and communication overhead by removing unnecessary model parameters and customizing the pruning rate. Our evaluation reveals significant deficits in current pruning methods in FL, where OPALA maintains its accuracy in the CIFAR-100 dataset above 58% while providing around 50% lower data transmission and around 25-14% lower training time.

The rest of this paper is organized as follows. Section II reviews the related work. Section III describes our approach to adaptive pruning aggregation in both server and client models. Section IV explores the simulation scenario and results. Finally, Section V concludes the paper and outlines the directions for future work.

## II. Related Work

Salehi *et al.* [15] proposed an FL framework to optimize sector selection in vehicular networks operating in the mmWave band called FLASH. This framework enhances the FL accuracy of local and global models by aggregating the client's model on the central server and pruning the local model clients. The framework adaptively prunes or retrieves model weights based on real-time data, ensuring that it maintains high performance and minimizes the loss in accuracy typically associated with model pruning.

Xu *et al.* [16] introduced an FL framework designed to address the limitations of resource-constrained IoT devices in big data analytics, called FL-PQSU. This framework minimizes computation, storage, and communication overhead during FL training by integrating structured pruning to reduce model size, weight quantization to enhance communication efficiency, and selective updating to prioritize relevant client contributions.

Jiang *et al.* [13] presented the PruneFL framework to address computational and communication challenges in resource-constrained edge devices. The distributed pruning method addresses data (statistical) and device (system) heterogeneity, including non-IID data distribution. PruneFL uses distributed parameter pruning to dynamically reduce model size throughout the training process, thereby cutting training time while maintaining model accuracy.

Yu *et al.* [17] proposed a method for compressing and accelerating Convolutional Neural Networks (CNNs) called Neuron Importance Score Propagation (NISP) by evaluating the importance of neurons across the entire network. Unlike traditional approaches that prune neurons layer by layer, NISP propagates neuron importance scores backward from the final response layer to earlier layers, ensuring that pruning decisions are globally optimized.

We analyze studies that apply pruning approaches in FL scenarios to understand their approach and identify areas for improvement to achieve satisfactory results. However, some approaches do not consider the structural method to prune the model, so the training time does not decrease considerably. In addition, existing approaches do not provide adaptability to adjust pruning according to the processing power or data of the device. Hence, the existing approach has some limitations that go beyond simple pruning. The analyzed approaches tackle these issues independently, failing to unify them under a single cohesive strategy.

## III. Adaptive Pruning Approach for pFL

This section introduces the OPALA approach to provide adaptive structured pruning based on processing time, training time, and data size in a pFL scenario, enabling clients with fewer data or slower processing capabilities to contribute to the global model with faster convergence time. OPALA is designed to improve model performance by allowing clients to participate in the process, even if they cannot send their parameters during training processing or do not have enough power to train. In the following, we introduce the system model and operational details of OPALA.

### A. Scenario overview

Figure 1 shows how the pFL generally works in a distributed scenario, where the typical FL approach consists of three essential steps in each round of communication $t$, namely client selection, training and aggregation. We consider a scenario involving an amount of $n$ devices $\mathcal{P} = \{p_1, \ldots, p_n\}$, where each device is uniquely identified by an index $i$ within the range $[1, n]$. Each communication round $t$ starts with the client selection step, which selects a subset of devices $\mathcal{C} \subseteq \mathcal{P}$ (clients) with the most valuable data samples, potentially excluding clients whose data no longer contribute significantly to the model's training. Afterward, the server transmits the current global model parameters $M_g^t$ for classification or prediction to all participating clients. The ML model comprises a Neural Network with $L$ layers denoted as $\mathcal{L} = \{l_1, l_2, \ldots, l_L\}$.

Each device $u_i$ has a local data set $D_i$ that varies significantly in size and feature and label distribution, typically showing non-IID characteristics. The local data set $D_i$ consists of a set of features $x_{k,i}$, where $D \in \{D_1, \ldots, \|D_i\|\}$, and each feature is associated with a label $y_{k,i}$. The specific device $u_i$ collects a total number of data samples represented by $|D_i|$. In the training step, each client $u_i$ improves the local model $M_g^t$ by updating the model parameters $\theta_i$ by minimizing a locally defined loss function $L_i(\theta)$ using a specified number of epochs

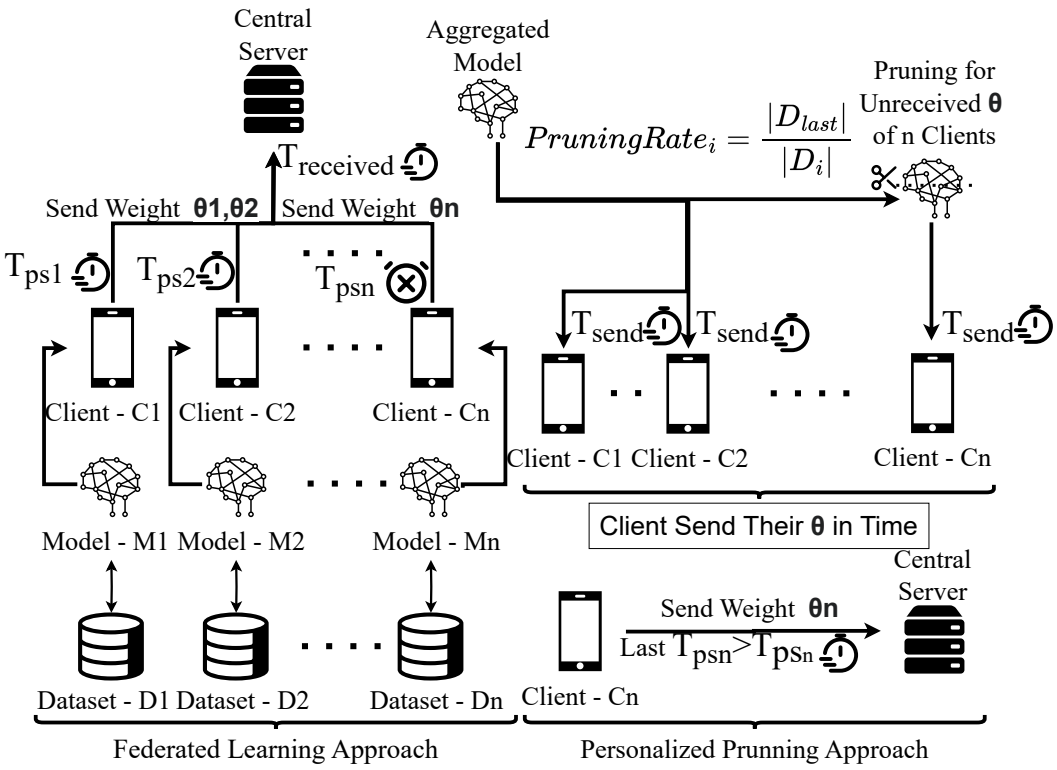



Fig. 1. Scenario overview for Personalized Pruning

$E$, as shown in Eq. 1. In this function, $f(x_{k,i};\theta)$ represents the prediction of the model for input $x_{k,i}$ based on the model parameters $\theta$, while $\ell$ denotes the loss function (cross-entropy or mean squared error, depending on the task).

$$L_i(\theta) = \frac{1}{|D_i|} \sum_{(x_{k,i},y_{k,i})\in D_i} \ell(f(x_{k,i};\theta), y_{k,i}), \tag{1}$$

We denote the total transmission time ($Tps$), given by Eq. 2, as the time required to send both the global model to the clients ($T_{send}$) and the time needed to receive the updated parameters ($T_{receive}$). This represents the entire transmission cycle from the server's perspective.

$$Tps = T_{send} + T_{receive} \tag{2}$$

After the local training, each client sends its updated parameters $M_i^t$ back to the Central Server for aggregation and refinement of the global model. Finally, the aggregate step completes the communication round by combining the model updates from selected clients $\mathcal{C}$ to produce the updated global model $M_g^t$. In this sense, the Adaptive Local Aggregation for Personalized Federated Learning (FedALA) [18] performs aggregation by incorporating client-side aggregation after server transmission. This results in more unique client models that can be fine-tuned models post-aggregation. Using the diversity of local datasets, FedALA improves the robustness of the global model while mitigating the influence of smaller or less representative datasets. The weighted averaging process ensures that clients with larger datasets contribute proportionally more to the global model update, effectively addressing imbalances in dataset sizes and promoting convergence, even in scenarios with non-IID data distributions. These steps were iterated over multiple communication rounds until convergence.

### *B. OPALA Operations*

ML models, such as Convolutional Neural Networks (CNN), contain many parameters that could be efficiently compressed, making the models more compact and easier to store or transmit. In this sense, adapting the global model to each client's local environment is crucial to improving efficiency in the pFL environment, and achieving that by removing unnecessary model parameters.

OPALA relies on the computational cost of each client, which is measured using the total floating point operations (FLOP) calculated according to the equation. 3. This represents the total FLOPs for a given client $C_i$, where $n_i$ is the number of local training samples, and $FLOPS(M_i^t)$ refers to the number of operations required to process a single input sample in the ML model $M_i^t$.

$$FLOPStot_i = n_i \times FLOPS(M_i^t) \tag{3}$$

Convolutional layers have a high computational cost due to the large number of multiplications and additions required per

layer. Thus, prune aims to reduce this overhead by eliminating the least significant parameters. In this sense, it is important to evaluate the importance of parameters based on the L2 norm $|\theta_i||_2$, computed based on Eq. 4. Specifically, the L2 norm $|\theta_i||_2$ measures the magnitude of each parameter in the model. Hence, lower L2 norm values in the parameters contribute less to the model performance, making them suitable for pruning first.

$$||\theta_i||_2 = \sqrt{\sum_j \theta_{i,j}^2} \tag{4}$$

The aggregation server computes the total transmission time ($Tps$) based on Eq. 2, representing the entire transmission cycle from the server's point of view. OPALA defines a predefined threshold (Ths) as a time limit to receive model updates. In this sense, clients with a total transmission time $Tps$ greater than Ths are included in a list of failed clients, and the server also includes clients that did not transmit model updates. This list of failed clients allows us to identify straggler clients that must reduce the model complexity to enable faster training and more efficient communication.

Eq. 5 determines the percentage of parameters to prune. We denote $|D_{last}|$ as the size of the data set of the last successfully received client model, serving as a reference to balance the pruning process between clients with different computational capacities. This ensures that clients with fewer resources perform selective parameter reduction while maintaining global model performance. Hence, OPALA is designed to improve the performance of clients who cannot send their parameters during training processing, or do not have enough power to train, considering the processing time, training time, and data size.

$$PruningRate_i = \frac{|D_{last}|}{|D_i|} \tag{5}$$

Algorithm 1 outlines the approach OPALA, which includes structured pruning. The algorithm begins with each client training its local model $M_i^t$ using the data set $D_i$, and attempting to send the updated parameters $\theta_i$ to the central server. If the transmission fails, the system marks the client as failed. Clients who fail to transmit or deliver model updates after a given threshold application are included in a list of failed clients to ensure that they have a minimum time. The Ths plays a crucial role in identifying straggler clients by setting a time limit for all clients to submit their updates. Adjusting this threshold ensures that client updates contribute to the global model, maintaining efficient communication and model accuracy for all clients. In the case of failed transmissions, the central server identifies the last successfully received client $C_{last}$ as a reference to calculate the pruning rate. Each failed client $C_i$ computes the pruning rate using the ratio based on the size of the data sets $|D_{\text{last}}|$ and $|D_i|$, as shown in Eq. 5. Finally, the system ranks the parameters according to their L2 norm. It prunes the least important ones to maintain model efficiency while reducing communication costs. Hence, this pruning approach is personalized for each client, making it adaptable to the unique characteristics of their data and processing capabilities, which further optimizes the overall system performance.

**Algorithm 1** OPALA Approach

```
for each client C_i ∈ C do
    Train local model M_i using local dataset D_i
    Try to send updated parameters θ_i to Central Server
    if transmission fails or pass application Ths then
        Mark C_i as a failed client
    end
end
if there are failed clients then
    Identify last successfully received client C_last
    for each failed client C_i do
        Compute pruning rate based on Eq. 5 Prune least
        important parameters based on L2 norn, which could
        fits to prune rate based on PruningRate_i
    end
end
```

## IV. Evaluation

This section presents the simulation setup used to evaluate the performance and efficiency of OPALA compared to the Baseline approaches. We first describe the simulated FL scenario, including the underlying framework, the characteristics of the database, and the simulation parameters. We also present the obtained results, focusing on the accuracy and loss metrics for the global model.

### A. Simulation Setup

In this paper, we consider PFLlib[1], a widely recognized open-source framework that supports multiple pFL methods. We chose PFLlib for its broad adoption within the research community and its flexible and user-friendly capabilities for developing and evaluating FL systems. Its architecture allows seamless customization and personalization of models for individual clients, making it well-suited to our objectives. Our simulation code can be found on Github [2]. Table I outlines the key simulation parameters used in our experiments.

The experiments consider the CIFAR-100 datasets, where each client performs one local epoch per round, ensuring consistent local training updates between all participants. Specifically, CIFAR-100 consists of 100 classes, each containing 600 images, resulting in a more fine-grained classification task. This data set is suitable for evaluating FL algorithms in scenarios requiring higher levels of granularity and complexity in image classification. We consider data with non-IID characteristics to introduce data heterogeneity, ensuring a realistic representation of data distribution as commonly expected in many Federated Learning (FL) applications. We

[1]PFLlib available at https://github.com/TsingZ0/PFLlib

[2]OPALA is available at https://github.com/OrenanM/FL---Prunning

TABLE I
SIMULATION PARAMETERS

| Description | Values |
|---|---|
| Number of Clients | 12 Clients |
| Training clients | 100% of total number of clients |
| Default dataset | CIFAR-100 |
| Local epochs | 1 epoch per round |
| Number of classes | 100 classes |
| Dirichlet distribution | 0.2 |
| Rounds | 100 |
| Base model size | 500mb |
| Used model | VGG |

used the Dirichlet distribution with a $\beta = 0.2$ concentration parameter, as described in [19].

We consider a widely recognized deep learning architecture (*i.e.*, VGG CNN) with a straightforward and consistent design. It comprises multiple stacked 3x3 convolutional layers followed by max-pooling layers, which gradually reduce spatial dimensions while increasing the depth of feature maps. This structure allows for efficient feature extraction. However, its heavy reliance on fully connected layers results in many parameters, making the model computationally intensive and susceptible to overfitting, especially when trained on smaller datasets. Despite these drawbacks, VGG remains a benchmark in computer vision due to its simplicity and effectiveness in many image classification tasks.

We compare OPALA's performance with two existing methods: single-shot network pruning (SNIP) [13], NISP [17], and Baseline FL. Specifically, SNIP identifies the most critical connections in a neural network by computing connection sensitivity before training, allowing for structured pruning while maintaining performance. NISP evaluates the importance of neurons by considering their contributions to the final output, enabling more fine-grained pruning at the neuron level. The Baseline FL is the traditional FL approach without pruning, but we introduce the Ths time. Finally, OPALA provides an adaptive structured pruning based on processing time, training time, and data size, enabling clients with fewer data or slower processing capabilities to contribute to the global model with faster convergence time, as introduced in Section III.

We consider four well-known metrics to evaluate system performance, namely average accuracy per round, training time per round (measured in seconds), loss, FLOPs, and Total Data Transmitted (TDT). These metrics provide a comprehensive assessment of model efficiency, communication overhead, and client participation, ensuring that our approach maintains competitive accuracy while optimizing data transmission and system performance.

### B. Results

Figure 2 shows the accuracy measurements of the three models over time. It is important to mention that CIFAR-100 has 100 distinct classes and more intricate class distinctions, naturally posing more significant challenges for classification or prediction to all participating clients. By analyzing the results, we can conclude that OPALA shows the best performance, reaching around 56% accuracy in just 100 seconds, demonstrating a faster convergence rate than NISP and SNIP. For example, NISP takes longer to improve and achieves an accuracy of approximately 52% at the 100-second mark. At the same time, SNIP shows the slowest progress, reaching only 30% accuracy and failing to improve after more than 40 seconds. This behavior occurs due to two factors in our approach. The first one is that the non-structural pruning of SNIP causes this pruning to delay processing and transmission. It also prevents too much processing time decay, which leads most clients not to reach the threshold this time. The second factor is that NISP does not consider a more leisure approach, which means that it is trying to optimize globally. However, it does not serve the clients well in FL because it already has an established global model with clients' aggregated parameters.

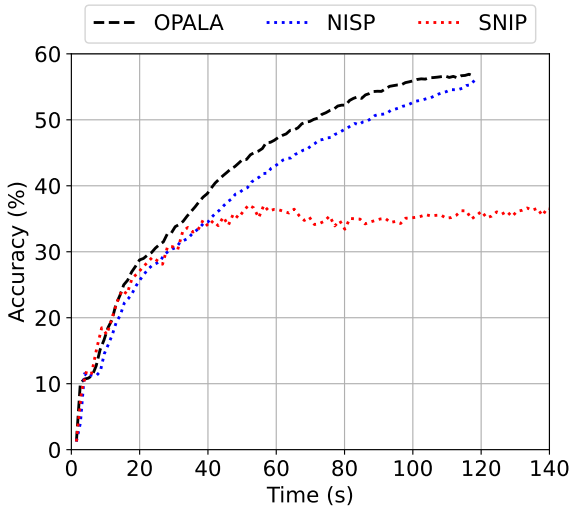



Fig. 2. Accuracy measurements

Figure 3 shows the loss measurements for OPALA, the NISP, and the SNIP approach in 100 rounds. The loss for OPALA starts around 4.5 and rapidly decreases, stabilizing below 2.0 after approximately 20 rounds. On the other hand, NISP starts with a higher loss value of around 4.0 and shows a gradual decrease, reaching a stable loss of about 2.5 by around 80. Finally, SNIP initially starts with a similar loss. However, it fluctuates more significantly, ending with a loss of about 3.0 after 100 rounds. The fluctuations in SNIP loss suggest a less stable learning process than the other methods. This instability of SNIP occurs because of unreached clients in the Ths threshold. These SNIP measures do not have all the evasion they need to make more progress in their loss measurements. On the other hand, we have NISP with a similar value to OPALA, which means that its structural methods help train the delivery of clients' models in the aimed threshold Ths as OPALA does. Hence, the OPALA approach demonstrates the fastest reduction in loss, stabilizing earlier than both NISP

and SNIP.

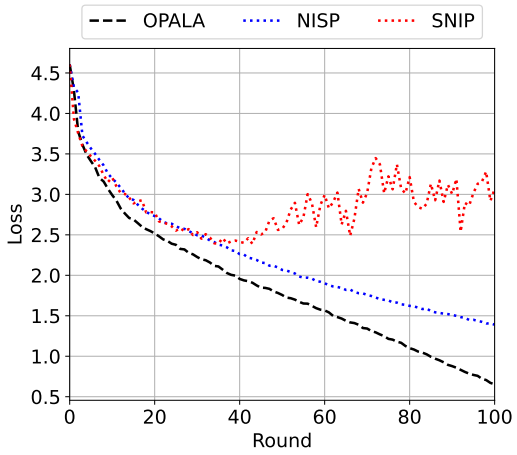



Fig. 3. Loss measurements

Figure 4 shows the FLOPS measurement of these three approaches during the presented scenario. OPALA did not perform best among these approaches despite achieving better accuracy and loss, reaching 122.15 MFLOPs. Meanwhile, NISP obtained the lowest value due to its more specialized layer removal method. Although it did not achieve the highest accuracy in the other evaluations, reaching only 121.72 MFLOPs, it shows that the calculation based on L2 is important to determine which neurons may be more dispensable in complexity. As for SNIP, it did not have as good an evaluation due to the lack of client delivery, a result expected from previous outcomes. Using non-structural pruning directly impacts the computational cost of achieving the same task.

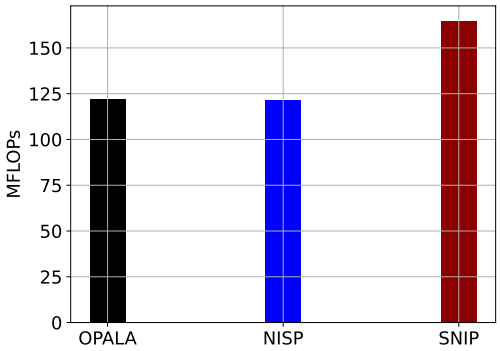



Fig. 4. MFLOPS measurements

s

Table **??** shows the improvements of OPALA compared to the Baseline and other pruning approaches under varying thresholds. Specifically, we evaluate the overall improvements achieved by our approach compared with Baseline FL in different Ths values, assessing the system's efficiency. By analyzing the results of the 1.47 threshold, we conclude that NISP performs similarly to OPALA with an accuracy of 56.66%, while SNIP bottlenecks with an accuracy of 36. 29%. At the 1.23 threshold, OPALA continues its improvement, reaching an accuracy of 55. 85% and reduce the training time to 1.10 seconds, outperforming the baseline by 83. 35%. NISP maintains its performance with 54.32% accuracy, while SNIP's performance remains significantly lower at 32.75%. These results demonstrate that OPALA consistently outperforms Baseline pruning and other pruning techniques with regard to accuracy and training efficiency at all thresholds. It shows its effectiveness in balancing model pruning and learning capacity.

At a threshold of 1.65, OPALA outperforms the Baseline by achieving a 58.94% accuracy, compared to the Baseline of 35. 49%, resulting in an improvement of 50.49%. Furthermore, OPALA reduces the training time (1.23 seconds) compared to the Baseline (1.53 seconds). Similarly, at a threshold of 1.47, OPALA's accuracy (56.66%) is significantly better than the Baseline's 29.22%, showcasing an improvement of 50.66% while maintaining a reduced training time (1.15 seconds). Finally, at a threshold of 1.23, OPALA continues to improve on the Baseline, with an accuracy of 55.85% compared to the Baseline of 30.46%, resulting in an improvement of 63.53% and a reduced training time (1.10 seconds). These results demonstrate that OPALA consistently outperforms Baseline accuracy and training efficiency on all thresholds.

At 1.65 seconds, OPALA and NISP aggregate data from 12 clients, resulting in a TDT of 294.264 GB each. In contrast, SNIP aggregates data from only four clients, leading to a lower TDT of 98.088 GB. At 1.47 s, OPALA and NISP continue with 12 clients, and the TDT drops slightly to 261.36 GB. SNIP, with only two clients, shows a much smaller TDT of 43.56 GB. At 1.23 s, OPALA and NISP still aggregate data from 12 clients, reducing the TDT to 217.488 GB. SNIP aggregates data from only one client, resulting in the lowest TDT of 18.124 GB, demonstrating the direct impact of client participation on the total data transmitted. Furthermore, OPALA has a 50-51% TDT reduction compared to the Baseline at thresholds of 1.65 and 1.47. At the 1.23 threshold, the shorter receiving time needed to prune is more effective, so it is more than the other two Ths, resulting in a 63.33% improvement over the Baseline.

## V. Conclusion

This paper introduces OPALA, an approach to improve the generalization capabilities of the pFL model by implementing an adaptive pruning aggregation of FL environments. OPALA improves the processing of the FL client by dealing with the bottleneck of delayed client instances, considering two crucial factors during the selection process: the model and the size of the client data. The evaluation results demonstrate that OPALA outperforms Baseline and state-of-the-art approaches, such as NISP and SISP. For example, OPALA improved by about 50% lower in transmission, and around 25-14% in training time. These results highlight the robustness and efficiency of OPALA, which ensures high model performance in FL

environments with pruned models. For future work, we plan to enhance OPALA by integrating additional compression techniques into our pruning approach, such as quantization, and considering client resource constraints to optimize data transmission in FL environments. Additionally, we aim to explore the applicability of OPALA across various FL scenarios and network metrics using ns-3 simulators.

## ACKNOWLEDGMENT

This work was partially sponsored by CNPq grant # 404186/2021-1, Amazon Foundation for Studies and Research Support (FAPESPA), the Brazilian Ministry of Science, Technology, and Innovations with resources from Law nº 8,248, of October 23, 1991, within the scope of PPI-SOFTEX, coordinated by Softex, and published under Arquitetura Cognitiva (Phase 3), DOU 01245.003479/2024-10. This work is also funded by PRR–Plano de Recuperação e Resiliência and by the NextGeneration EU funds at Universidade de Aveiro, through the scope of the Agenda for Business Innovation "ATT–Agenda Mobilizadora Acelerar e Transformar o Turismo" (Project no. 47 with the application C645192610-00000060).

## REFERENCES

[1] F. M. A. Khan, H. Abou-Zeid, and S. A. Hassan, "Deep compression for efficient and accelerated over-the-air federated learning," *IEEE Internet of Things Journal*, 2024.

[2] P. Kairouz, H. B. McMahan, B. Avent, and et al., "Advances and open problems in federated learning," *Foundations and trends® in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[3] B. Hu, X. Huang, K. Zhang, F. Wu, C. Sun, T. Cui, and Y. Zhang, "Enhanced federated reinforcement learning for mobility-aware node selection and model compression," in *IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2023, pp. 158–163.

[4] X. Zhu, J. Wang, W. Chen, and K. Sato, "Model compression and privacy preserving framework for federated learning," *Future Generation Computer Systems*, vol. 140, pp. 376–389, 2023.

[5] F. Haddadpour, M. M. Kamani, A. Mokhtari, and M. Mahdavi, "Federated learning with compression: Unified analysis and sharp guarantees," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2021, pp. 2350–2358.

[6] B. Martins, A. M. Souza, D. Rosário, C. A. Astudillo, E. Cerqueira, and L. Villas, "Partial training mechanism to handle the impact of stragglers in federated learning with heterogeneous clients," in *IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2024, pp. 1–6.

[7] N. Razmi, B. Matthiesen, A. Dekorsy, and P. Popovski, "On-board federated learning for satellite clusters with inter-satellite links," *IEEE Transactions on Communications*, 2024.

[8] C. Zhang, W. Zhang, Q. Wu, P. Fan, Q. Fan, J. Wang, and K. B. Letaief, "Distributed deep reinforcement learning based gradient quantization for federated learning enabled vehicle edge computing," *IEEE Internet of Things Journal*, 2024.

[9] J. Wu, L. Wang, and Y. Wang, "An improved cnn-lstm model compression pruning algorithm," in *Advances in Natural Computation, Fuzzy Systems and Knowledge Discovery*. Springer, 2022, pp. 727–736.

[10] M. Ye, X. Fang, B. Du, P. C. Yuen, and D. Tao, "Heterogeneous federated learning: State-of-the-art and research challenges," *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–44, 2023.

[11] A. M. de Souza, F. Maciel, J. B. da Costa, L. F. Bittencourt, E. Cerqueira, A. A. Loureiro, and L. A. Villas, "Adaptive client selection with personalization for communication efficient federated learning," *Ad Hoc Networks*, vol. 157, p. 103462, 2024.

[12] X. Wang, H. Jiang, M. Mu, and Y. Dong, "A trackable multi-domain collaborative generative adversarial network for rotating machinery fault diagnosis," *Mechanical Systems and Signal Processing*, vol. 224, p. 111950, 2025.

[13] Y. Jiang, S. Wang, V. Valls, B. J. Ko, W.-H. Lee, K. K. Leung, and L. Tassiulas, "Model pruning enables efficient federated learning on edge devices," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 12, pp. 10 374–10 386, 2022.

[14] N. Yan, Y. Li, J. Chen, X. Wang, J. Hong, K. He, and W. Wang, "Efficient and straggler-resistant homomorphic encryption for heterogeneous federated learning," in *IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2024, pp. 791–800.

[15] B. Salehi, D. Roy, J. Gu, C. Dick, and K. Chowdhury, "Flash-and-prune: Federated learning for automated selection of high-band mmwave sectors using model pruning," *IEEE Transactions on Mobile Computing*, 2024.

[16] W. Xu, W. Fang, Y. Ding, M. Zou, and N. Xiong, "Accelerating federated learning for iot in big data analytics with pruning, quantization and selective updating," *IEEE Access*, vol. 9, pp. 38 457–38 466, 2021.

[17] R. Yu, A. Li, C.-F. Chen, J.-H. Lai, V. I. Morariu, X. Han, M. Gao, C.-Y. Lin, and L. S. Davis, "Nisp: Pruning networks using neuron importance score propagation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.

[18] J. Zhang, Y. Hua, H. Wang, T. Song, Z. Xue, R. Ma, and H. Guan, "Fedala: Adaptive local aggregation for personalized federated learning," in *AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, 2023, pp. 11 237–11 244.

[19] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 10 713–10 722.